

Get In Line

Team Name: Team 12

Team Members:

Kyle Albuquerque

Angela Lopez

Disha Shetty

Neel Vavivapalli

Supervisor:

Professor Sridhar Alagar

Project Coordinator:

Adarsh Gella

**Erik Jonsson School of Engineering and Computer Science
University of Texas at Dallas**

Date: May 5, 2026

Table of Contents

Executive summary

- 1. Introduction**
 - 2. Project Specification & Management**
 - 3. System Architecture & High-Level Design**
 - 4. Implementation & AI Orchestration**
 - 5. System Performance & Validation**
 - 6. Lessons Learned**
 - 7. Conclusion & Future Work**
- References**
 - Acknowledgements**

Executive Summary

Get In Line is a scalable waitlist management system designed to optimize how businesses handle event attendance, queue flow, and capacity constraints. The platform enables businesses to create and manage events while allowing users to join waitlists and receive dynamically calculated wait time estimates. By supporting multiple event types—including attendance-based, queue-based (single and multi-queue), and capacity-limited events—the system is adaptable to a wide range of real-world use cases such as restaurants, pop-ups, and high-traffic venues.

The system addresses key inefficiencies in traditional waitlist management, including inaccurate time estimates, lack of real-time updates, and poor handling of network disruptions. To solve this, **Get In Line** implements a local-first, hybrid synchronization architecture that ensures continuous usability even during intermittent connectivity. The frontend maintains operational state using `localStorage`, allowing users and staff to continue interacting with the system offline, while background synchronization reconciles data with the cloud once connectivity is restored.

The architecture is composed of three primary layers: a Vite/React frontend built with TypeScript, a Node.js/Express backend acting as the central orchestrator, and a FastAPI service dedicated to processing waitlist heuristics and generating wait time predictions based on real-time user activity. Supabase serves as the centralized data store and source of truth, ensuring consistency across the system. This separation of concerns enables modular development, improved maintainability, and the ability to scale individual components independently.

Performance validation using k6 demonstrated strong system responsiveness and stability under load. The system maintained average API latencies of approximately 1.1–1.2 ms, with 95th percentile latencies remaining below ~3 ms, even as concurrency scaled up to hundreds of virtual users. Throughput increased proportionally with load, reaching up to ~375 requests per second without significant degradation, indicating efficient request handling and strong scalability characteristics.

The project also highlights the integration of AI-assisted development workflows, where AI tools were leveraged for planning, code generation, debugging, and documentation. While not part of the runtime system, these tools accelerated development and required structured human oversight to ensure correctness and alignment with project goals.

Overall, **Get In Line** delivers a robust, responsive, and scalable solution for modern waitlist management, combining real-time interaction, offline resilience, and high-performance system design to improve both business operations and user experience.

1. Introduction

1.1 Motivation & Problem Statement

Waitlist management is a common challenge for businesses hosting events or managing customer flow. Traditional systems often rely on manual processes or static estimates, which can lead to inefficiencies, inaccurate wait times, and poor customer experience. Businesses lack flexible tools to handle different types of events, such as general attendance, structured queues, or capacity-limited environments.

The motivation behind **Get In Line** was to create a unified, adaptable system that simplifies waitlist management while improving transparency and accuracy for users.

1.2 Objectives, Scope, and Goals Achieved

Objectives:

- Enable businesses to create and manage events
- Allow users to join waitlists seamlessly
- Provide dynamic wait time estimations
- Support multiple event types (attendance, queue, capacity)

Scope:

- Web-based application with frontend and backend integration
- Event creation, editing, and participation
- Waitlist tracking and time estimation

Goals Achieved:

- Successful implementation of event creation and management
- Functional waitlist system with calculated wait times
- Support for three event types
- Integration with a database for persistent data storage

1.3 Proposed Solution

The proposed solution is a full-stack web application that integrates a frontend interface with backend services and a database. Businesses can create events with specific configurations, while users can browse and join these events. The system calculates wait times using predefined equations based on queue structure and capacity constraints.

The application ensures real-time interaction between users and event organizers through API communication and database synchronization, resulting in a cohesive and responsive system.

1.4 Key Highlights

- Implementation of multiple event types (attendance, queue, capacity)
- Dynamic wait time calculation without reliance on external AI models
- Integration with Supabase for data management

- High-performance API responses validated through load testing
- Use of AI tools during development for planning, debugging, and documentation

2. Project Specification & Management

2.1 The Project Specification (The AI's Blueprint + Link)

A structured guideline document was generated at the beginning of the project and used as the foundation for development. This document served as the primary instruction set for guiding AI-assisted development and maintaining consistency across the team.

Link:

https://github.com/zap-bit/Waitlistmanagementappcopy/blob/main/Documentation/FINAL_PROJECT_SPEC.md

2.2 Maintaining the "Single Source of Truth"

The project specification was continuously updated and referenced throughout development to ensure alignment between team members and AI-generated outputs. This approach helped maintain consistency and reduced discrepancies in implementation.

2.3 Key Milestones

- Creation of initial project specification and guidelines
- Development of a functional MVP with core features
- Integration of frontend, backend, and database
- Performance testing and validation using k6

2.4 Team Roles & Leader Rotation

The team consisted of four members with clearly defined technical responsibilities. Kyle was primarily responsible for database design and management, including schema design, data modeling, Supabase integration, and stress testing. Disha led the local storage to Supabase migration, backend development, defining and implementing API endpoints, business logic, and server-side architecture, while also contributing to frontend implementation including staff dashboard functionality, event management, third-party integrations, and UI design including custom hand-drawn icons, frame-by-frame animation, and branding assets. Angela focused on frontend development, building the user interface and ensuring a responsive, mobile-friendly experience using React. Neel specialized in AI-related tasks, supporting the team through AI-assisted development processes such as code generation, debugging, and implementation guidance. All four members contributed collectively to project planning, documentation, and architectural decision-making throughout the development lifecycle.

Leadership responsibilities were rotated among team members every three weeks, ensuring that each member had the opportunity to guide project direction, coordinate tasks, and oversee progress. This rotation encouraged shared accountability and provided each team member with

experience in both technical and organizational roles.

3. System Architecture & High-Level Design

3.1 System Overview

The system is designed as a multi-layered, hybrid architecture that prioritizes real-time responsiveness, offline resilience, and scalability. It follows a decoupled client-server model consisting of three primary components.

The frontend, built with Vite and React using TypeScript, serves as the primary interface for both guests and staff. It implements a local-first design, allowing core functionality such as waitlist management and table updates to persist through localStorage even during network disruptions.

The primary backend is implemented using Node.js with Express and acts as the central orchestrator. It handles authentication, event management, staff operations, and synchronization logic while serving as the gateway between the frontend and the database.

A secondary FastAPI service functions as the intelligence layer, responsible for processing waitlist heuristics and generating dynamic wait time predictions based on real-time user activity signals ("pings").

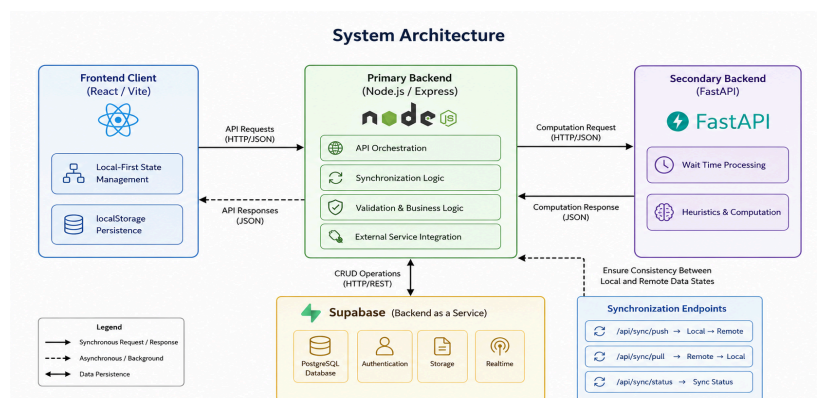
Supabase serves as the centralized data store and source of truth, managing authentication, event configurations, and persistent waitlist data. The system incorporates synchronization mechanisms to reconcile local and remote data, ensuring consistency across distributed components.

3.2 Architecture Diagrams

The system architecture consists of three main layers:

- Frontend client (React/Vite) with local-first state management and localStorage persistence
- Primary backend (Node.js/Express) handling API orchestration and synchronization
- Secondary backend (FastAPI) for wait time processing and heuristics

Data flows from the frontend to the Node.js API, which interacts with Supabase for persistence and communicates with the FastAPI service for computation. Synchronization endpoints ensure consistency between local and remote data states.



3.3 Key Architectural Decisions and Trade-offs

A key architectural decision was adopting a local-first design approach on the frontend. By persisting state in `localStorage` and performing optimistic updates, the system ensures continuous usability even during network instability. This improves user experience but introduces additional complexity in synchronization and conflict resolution.

The system separates responsibilities across two backend services: a Node.js/Express orchestrator for core application logic and a FastAPI service for wait time calculations. This separation allows independent scaling and optimization of computational logic but increases system complexity and coordination overhead.

Supabase was selected as the centralized database due to its real-time capabilities and ease of integration. A "SERVER_WINS" conflict resolution strategy was implemented to maintain data integrity during synchronization, prioritizing consistency over local changes.

The primary trade-off in this architecture is increased complexity due to multiple services and synchronization logic. However, this is justified by improved reliability, modularity, and scalability, especially in environments with intermittent connectivity.

4. Implementation & AI Orchestration

4.1 Technologies Used

- Frontend: Vite + React with TypeScript (local-first architecture with `localStorage` persistence)
- Backend: Node.js with Express (primary API and orchestrator), FastAPI (waitlist heuristics and prediction service)
- Database: Supabase (centralized data storage and authentication)
- Hosting/Deployment: Vercel (frontend), Render (backend services)

4.2 AI Build Strategy

AI tools were used extensively during development for code generation, debugging, planning, documentation, and organization. The team first generated a structured guideline and then built the system around it, ensuring consistent outputs from AI tools.

4.3 Human-in-the-Loop Interventions

During development, manual intervention was required to correct AI-generated issues. These included cases where AI would remove necessary code or modify unrelated sections, requiring developers to review, debug, and restore functionality.

4.4 Codebase

The project codebase is organized into a multi-service architecture, consisting of a frontend client, a primary backend API, and a secondary service for waitlist processing. The frontend is built using Vite and React with TypeScript, while the backend is split between a Node.js/Express API and a FastAPI service for wait time heuristics.

Version control is managed through GitHub, where the repository contains the full project source code, including frontend, backend, and configuration files. The repository also includes a README.md file that provides setup instructions, environment configuration details, and an overview of the system architecture.

Merged Repository Link: <https://github.com/zap-bit/Waitlistmanagementappcopy>

5. System Performance & Validation

5.1 Testing and Validation

Testing and validation were conducted using k6 to simulate real-world load conditions and evaluate system performance under stress. The testing approach focused on measuring API latency, throughput, and response consistency as concurrency increased.

Tests were executed using virtual users (VUs) performing repeated API requests with controlled iteration pacing. Metrics collected included request duration, latency percentiles (p90, p95), and throughput. Concurrency levels were scaled progressively to observe system behavior under increasing load.

5.2 Technical Performance Metrics

- Average latency: ~1.1–1.2 ms
- Median latency: ~0.95–1.04 ms
- p90 latency: ~1.7–2.06 ms
- p95 latency: ~2.23–2.59 ms
- Maximum latency: ~36–81 ms
- Throughput: ~225–375 requests per second
- Virtual Users tested: 100–500 concurrent users

These results demonstrate low latency, high consistency, and stable performance under increasing load conditions.

6. Lessons Learned

6.1 Insights Gained

The project highlighted the effectiveness of combining traditional software development with AI-assisted workflows. AI tools significantly improved productivity in planning, coding, and documentation, but required careful oversight.

6.2 Lessons from Challenges

Key challenges included managing AI limitations during development, handling constraints such as AI credit usage, and resolving synchronization complexities between localStorage and Supabase.

Implementing a local-first architecture introduced additional difficulty in maintaining data consistency, particularly during offline scenarios and conflict resolution. This required the implementation of synchronization endpoints and a defined conflict resolution strategy ("SERVER_WINS").

AI-related issues also required manual intervention, as generated code would occasionally modify unrelated components or remove necessary logic. These challenges were addressed through careful validation, iterative debugging, and maintaining a structured development guideline.

6.3 Skills Developed

The team developed skills in full-stack development, API integration, database management, performance testing, and AI-assisted engineering practices.

6.4 Individual Reflection

Kyle Albuquerque:

Throughout the development of **Get In Line**, my primary contributions focused on the backend and database integration, particularly working with Supabase and ensuring reliable communication between the backend and the database. I was involved in implementing core backend functionality such as event creation, table editing, attendance tracking logic, assigning and seating parties, as well as handling data flow between services. While some of these implementations were later adapted or expanded by the team, my work helped establish the foundational structure for backend operations.

A significant portion of my work involved designing and managing the Supabase

database. This included creating and refining the database schema, writing efficient queries, and implementing real-time updates to keep the system synchronized. Through this process, I developed a much stronger understanding of how to structure data effectively and optimize queries for performance. On the backend side, I gained experience working with API calls, which was something I had not worked with extensively prior to this project.

One of the main challenges I encountered was ensuring proper synchronization between the frontend and backend. Inconsistent data flow often caused logic errors, especially when handling real-time updates and state changes. Debugging these issues required careful tracing of how data moved through the system and identifying where mismatches occurred. This highlighted the importance of tight integration between frontend and backend early in development.

In terms of performance and validation, I learned how to use k6 for stress testing, which was a new tool for me. Using the VSCode extension, I was able to simulate concurrent users and analyze system performance metrics such as latency and throughput. This experience gave me practical insight into how backend systems behave under load and how to interpret performance data to evaluate system stability.

AI tools played a major role in my workflow, particularly for generating code, assisting with documentation, and organizing development tasks. However, they also introduced challenges. In several cases, AI-generated changes modified unrelated parts of the codebase or introduced unintended bugs, requiring careful review and manual correction. This reinforced the importance of not relying on AI blindly and always validating generated outputs.

If I were to approach this project again, I would prioritize connecting the frontend and backend much earlier in the development process. Delaying integration made it harder to identify issues related to data flow and synchronization, which could have been caught sooner through iterative testing.

Overall, this project significantly improved my skills in database design, query optimization, and backend development. It also gave me practical experience with API integration, performance testing, and working within a collaborative, AI-assisted development environment.

Angela Lopez:

Throughout the development of Get In Line (GIL), my primary focus was on the frontend user experience and the critical task of ensuring seamless integration between the UI and the backend services. My role bridged the gap between the initial conceptual designs and the functional application, ensuring that the features we envisioned were both intuitive for the user and technically sound in their execution.

I began by leading the design phase in Figma, where I developed high-fidelity prototypes for the waitlist management interface. This process was more than just aesthetic; it required planning the user flow for event organizers and attendees to ensure the complex logic of table editing and party seating felt effortless. One of my main responsibilities was translating these designs into a responsive frontend that could handle real-time data updates. A significant portion of my technical work involved establishing the connection to the Supabase backend to ensure that when a user performed an action, such as joining a waitlist or updating a party's status, the information synchronized accurately and immediately across the database.

Working within an Agile framework with weekly sprints was instrumental in keeping our frontend and backend teams aligned. We used Jira to track our progress, which helped me prioritize which UI components needed to be functional first to support Kyle's backend development. This collaborative environment also involved using AI tools for rapid prototyping and troubleshooting frontend logic. Like my teammates, I found that while AI was helpful for generating boilerplate code, I had to be diligent in reviewing the output to ensure it adhered to our specific architectural requirements and didn't break existing integration points.

Reflecting on the semester, this project deepened my understanding of the full-stack lifecycle. If I were to approach this again, I would prioritize establishing the frontend-backend connection even earlier to catch state-management edge cases sooner. Overall, the experience of building GIL has been invaluable in refining my skills in UI/UX design, frontend development, and the complexities of building a synchronized, data-driven application.

Disha Shetty:

Working on the GIL (Get In Line) Waitlist Management Application over eleven weeks was a defining experience in my academic journey, offering a comprehensive look at the intersection of system architecture, technical execution, and project leadership. Serving as project coordinator under a rotating leadership structure, I contributed across the full development lifecycle from initial architecture through deployment and final polish.

At the onset, I focused on establishing a resilient system foundation by designing the architecture using draw.io and Gemini, mapping out core modules for waitlist management, capacity tracking, and AI-based wait time prediction. To ensure a smooth transition from architectural design to development, I authored the initial backend boilerplate and established Swagger/OpenAPI specifications using a contract-first approach. By defining clear endpoint structures, request-response schemas, validation rules, and error handling protocols early on, I reduced friction between frontend and

backend teams and prevented many of the integration issues that typically arise during early-stage development.

As development intensified, I leveraged a suite of tools to maintain system quality and velocity. I used Codex for boilerplate efficiency and Claude for subsequent integrations and planning assistance. A significant breakthrough came through using [Atelier](#), a memory-graph application I built myself. By utilizing its graph capabilities, I diagnosed a persistent alignment issue that appeared to be a frontend bug but was actually a deep-seated architectural flaw. Identifying the root cause allowed us to fix the system structurally rather than applying superficial patches. I also selectively used 21st.dev for UI smoothness while maintaining a high standard for core functionality.

Authentication was handled through Supabase, which also served as our primary database. I established the complete Supabase integration across both the frontend and backend, setting up the foundational connection layer that the team subsequently built upon, and helped plan resilience strategies for API failures and concurrent user handling through relevant middleware. On the backend, I evaluated and deployed using Render, ensuring the backend remained stable and accessible throughout the testing and demo phases.

A substantial portion of the final phase was dedicated to the application's visual identity and user experience. I designed and finalized the GIL logo, hand-drew custom icons for each event type — capacity-based, table-based, and attendance tracker — and created a frame-by-frame UI animation entirely by hand without any generative tools, a lengthy process that gave a more polished finish. I then integrated all custom assets into the frontend, replacing generic Lucide icons with our branded visuals and reformatting the UI layout to accommodate them. I also added the favicon to the web app. This work reinforced how directly design decisions affect implementation complexity, and how a polished interface significantly elevates the overall product.

Beyond visual work, I conducted end-to-end testing of various user flows, uncovering and resolving bugs including table count reconciliation issues when switching between past events. I integrated PostHog analytics into the frontend, handling SDK installation, environment variable configuration, and React provider setup. Beyond immediate usage tracking, PostHog's behavioral data has direct future value — the insights it captures on user flow and wait patterns could be used to fine-tune the AI wait time prediction model planned in our system architecture. I also researched Vercel analytics as an alternative, and investigated solutions for two key bottlenecks: local storage limitations addressed through IndexedDB, and inefficient single-storage Supabase queries addressed through Redis caching.

The later weeks also involved serious scalability planning. I documented strategies for supporting concurrent users through horizontal scaling, rate limiting, and

load balancing. I researched QR code and event code validation systems, planned offline mode architecture around local storage constraints, and outlined the infrastructure needed to move the application from prototype to production scale.

Reflecting on the process, I learnt how important critical disciplined branch management and consistent team communication are in collaborative development. In hindsight, addressing scalability and offline-first architecture earlier in the lifecycle would have further streamlined final integration.

Ultimately, the GIL project strengthened my ability to lead through complexity, design for scale, and build software that values both the precision of its architecture and the quality of its user experience. Through architecture design, API development, scalability planning, documentation, team coordination, custom visual design, and analytics integration, I gained valuable experience that will continue to influence my future projects and career goals.

Neel Vavivapalli:

Working on the GIL (Get-In-Line) Waitlist Management Application over the past eleven weeks has been a masterclass in full-stack engineering and agile collaboration. My role centered on bridging the gap between high-level architectural requirements and the granular execution of a real-time system, a process that demanded equal parts technical precision and clear communication.

Single Source of Truth & Team Alignment

One of the most vital lessons I learned was the necessity of a Single Source of Truth (SSOT). Early in the project, I realized that without a centralized definition of our data structures, the frontend and backend would inevitably drift apart. I spearheaded the effort to align the team by documenting a rigorous three-pillar architecture. By maintaining a centralized schema within Supabase and ensuring our FastAPI service and Next.js frontend strictly adhered to the same TypeScript interfaces, we eliminated "integration debt." This alignment ensured that when a database column changed, the impact was understood across the entire stack, from the heuristic wait-time logic to the UI components.

Real-Time Optimization & Backend Efficiency

Developing a live application forced me to confront the hidden "costs"—both financial and computational—of real-time data. Initially, our backend functions were taxing the system with frequent, unoptimized calls. I focused on optimizing our FastAPI logic to reduce overhead, ensuring that our heuristic wait-time models and database queries

were efficient enough to handle concurrent users without lag.

I specifically looked into:

- Database Normalization: Ensuring our Supabase tables were structured to prevent redundant data processing.
- Logic Refinement: Transitioning our WaitTime calculations from simple linear approximations to more robust backend-driven heuristics that only triggered when essential.
- Stress Testing Research: Preparing for scalability by researching tools like K6 and JMeter to simulate high-traffic scenarios and identify bottlenecks before they reached production.

Client Interaction & The Feedback Loop

A unique aspect of this project was treating our Professor and TA as simulated clients. Presenting our progress during weekly demos taught me how to translate technical milestones into "client value." I learned that a client doesn't just want to hear that a function works; they want to see how it solves the user's problem. This environment sharpened my ability to take constructive feedback and pivot our development priorities—such as refining our multi-threaded math server logic—to meet "stakeholder" expectations.

Technical Growth & Development Practices

This project expanded my technical repertoire significantly. I moved beyond basic coding into sophisticated system orchestration using a diverse toolset:

- Backend & AI: I implemented four distinct AI strategies (Regression, Classification, Naive Bayes, and Reinforcement Learning) to tackle the complexities of waitlist prediction.
- Documentation: I learned that code is only as good as its documentation. Using Swagger/OpenAPI to document our API endpoints became a cornerstone of our development process, allowing my frontend partners to work independently of my backend progress.

Deployment and Live Updates

Deploying a live app on platforms like Render was an eye-opening experience. I learned the high stakes of "hot-fixing" a production environment. Moving from a local development environment to a live URL required a disciplined approach to environment variables and branch management. I adopted better development practices, such as:

- **Modular Code:** Writing reusable functions so that updates to the AI service didn't break the core waitlist logic.
- **Version Control:** Using Git not just as a backup, but as a communication tool through descriptive commit messages and PR reviews.

Reflection on Communication

Perhaps the most profound growth occurred in my communication style. I realized that being a good developer isn't just about writing clean code; it's about making that code—and the logic behind it—understandable to others. Whether it was explaining a heuristic model to a non-technical teammate or documenting a bug for the database lead, I learned to simplify complex concepts without losing their technical integrity.

Ultimately, the GIL project transformed my approach to software engineering. It taught me that a successful application is the result of disciplined architecture, a constant eye on backend performance, and a team that speaks the same technical language. These insights into real-time system design and collaborative development will be the foundation of my professional career as a computer science student and developer.

7. Conclusion & Future Work

7.1 Summary of Key Accomplishments

The project successfully delivered a functional waitlist management system that meets its core objectives. It provides flexible event management, dynamic wait time estimation, and strong system performance, demonstrating the effectiveness of the proposed solution.

7.2 Future Enhancements

Future improvements will focus on enhancing scalability and system robustness. The current backend is deployed as a single-instance service, which limits horizontal scalability. Expanding to a multi-instance backend architecture with load balancing would allow the system to handle significantly higher traffic volumes.

Additional enhancements include implementing caching layers (such as Redis) to reduce database load, introducing database read replicas for improved performance, and further optimizing synchronization mechanisms. Containerization and orchestration tools could also be adopted to support large-scale deployments across multiple

environments.

References

Anthropic. (2025). *Claude* [AI assistant]. Used for code auditing, integration planning, and development support. <https://claude.ai>

Google. (2025). *Gemini* [AI model]. Used for architecture ideation, system design, and implementation guidance. <https://gemini.google.com>

GitHub, Inc. (2025). *GitHub* [Version control platform]. Used for repository management, team collaboration, and version control. <https://github.com>

Grafana Labs. (2025). *k6* [Load testing tool]. Used for stress testing, performance validation, and system analysis. <https://k6.io>

Figma, Inc. (2025). *Figma* [Design and prototyping tool]. Used for UI/UX design and interface planning. <https://figma.com>

ibis inc. (2025). *Ibis Paint X* [Digital illustration app]. Used for creating custom icons and hand-drawn visual assets. <https://ibispaint.com>

JGraph Ltd. (2025). *Draw.io* [Diagramming tool]. Used for system architecture design and technical diagramming. <https://draw.io>

Meta Open Source. (2025). *React* [JavaScript library]. Used for building the frontend user interface. <https://react.dev>

Microsoft. (2025). *GitHub Copilot* [AI coding assistant]. Used for code suggestions and development efficiency. <https://github.com/features/copilot>

Microsoft. (2025). *TypeScript* [Programming language]. Used for type-safe frontend and backend development. <https://typescriptlang.org>

OpenAI. (2025). *ChatGPT* [AI language model]. Used for planning, code generation, debugging, and documentation support. <https://openai.com/chatgpt>

OpenAI. (2025). *Codex* [AI coding model]. Used for boilerplate code generation and development efficiency. <https://openai.com/codex>

OpenAPI Initiative. (2025). *Swagger/OpenAPI Specification* [API documentation standard]. Used for defining endpoint structures, request-response schemas, and validation rules. <https://swagger.io>

PostHog, Inc. (2025). *PostHog* [Product analytics platform]. Used for tracking user behavior, events, and application performance insights. <https://posthog.com>

Gamma Technologies. (2025). *Gamma* [AI presentation tool]. Used for presentation design and content organization. <https://gamma.app>

Render. (2025). *Render* [Cloud hosting platform]. Used for backend deployment and production hosting. <https://render.com>

Shetty, D. (2025). *Atelier* [Memory-graph application]. Custom-built tool used for diagnosing architectural flaws through graph-based memory analysis.

Supabase, Inc. (2025). *Supabase* [Open-source backend platform]. Used for database management, authentication, and real-time data integration. <https://supabase.com>

21st.dev. (2025). *21st.dev* [UI component platform]. Used for frontend UI smoothness and component refinement. <https://21st.dev>

Vercel, Inc. (2025). *Vercel* [Frontend deployment platform]. Used for frontend deployment and analytics research. <https://vercel.com>

Acknowledgements

We would like to express our sincere gratitude to Dr. Alagar and Adarsh Gella for their invaluable guidance and support throughout the development of this project.

We are especially thankful for the consistent weekly meetings, where they provided insightful feedback, helped us work through technical challenges, and guided us toward effective solutions whenever we encountered obstacles. Their mentorship played a major role in shaping both the direction and quality of our work.

We also greatly appreciate the resources and recommendations they shared with us, particularly regarding stress testing, system documentation, and project organization. These materials helped us better understand industry practices and improve the overall reliability and professionalism of our system.

In particular, we would like to thank Adarsh Gella for going above and beyond by organizing a mock presentation session for our group. This opportunity allowed us to practice presenting our project, receive constructive feedback, and improve both our technical communication and confidence before the final presentation.

Their encouragement, availability, and dedication to helping students succeed created a highly supportive learning environment throughout the course. We are truly grateful for their time, patience, and mentorship, all of which contributed significantly to the successful completion of this project.